

C# (.NET)

Es el lenguaje de la plataforma más usado actualmente, se pueden desarrollar distintos tipos de aplicaciones. Es multiparadigma (soporta POO, programación funcional es decir pasar funciones como parámetros de otras, retornar funciones, expresiones lambda...). Lenguaje fuertemente tipado.

Fundamentos del lenguaje C#

Tipos de Datos

Tipo primitivo (WRAPPER)	Estructura.NET
byte	System.Byte
short	System.Int16
int	System.Int32
long	System.Int64
float	System.Float
double	System.Double
decimal	System.Decimal
boolean	System.Boolean
char	System.Char
String	System.String

Un tipo wrapper es un tipo de datos que no necesita usar constructor para inicializar valores. El objeto se instancia en el momento de darle valor.

Los tipos enteros

- byte: 1 bytes sin signo
- short se almacena en 2 bytes y su rango es $-32768 - 32767$
- int se almacena en 4 bytes y su rango es $-2147483646 -$
- Long se almacena en 8 bytes (Long.MaxValue) (Long.MinValue)

Los Tipos Reales

- float se almacena en 4 bytes
- double se almacena en 8 bytes
- decimal es nuevo en .NET, está compuesto de 12 bytes de valores enteros combinados con dos valores adicionales que controlan la ubicación de la coma decimal y del signo del valor numérico. Su precisión es mayor que la de double y se puede jugar con el lugar que ocupa la coma según interese.

char

El conjunto de caracteres predeterminados de VB.NET es Unicode. Un char es por tanto un valor de dos bytes.

Tiene algunos métodos muy útiles para validaciones: IsDigit ,IsControl, Isletter...

Están todos sobrecargados, pueden recibir un solo char o una cadena y una posición de la misma.

```
char le  
Char.IsDigit(le);  
Char.IsLetter("12345678S", 8 );
```

String

Es el único tipo básico que es una referencia. Representa cadenas de longitud variable. Puede almacenar desde 0 hasta unos 2.000 millones de caracteres unicode.

Object

Tipo de dato que puede contener cualquier cosa, un objeto, un integer,....

Tipos por valor y Tipos por referencia

.NET tiene sólo dos tipos principales de variables: valor y referencia.

- ❑ Los tipos de valor representan el almacenamiento de datos simples ubicado en la pila, estos son los que se suelen denominar tipos básicos de datos.

- ❑ Los tipos de referencia están basados en clases y heredan su implementación.

Se tratan de manera diferente en las sentencias de asignación y en la gestión de memoria. Cuando los datos se asignan a un tipo de valor, estos se almacenan en la propia variable que está situada en la *pila*. Cuando los datos se asignan a un tipo de referencia, en la variable se almacenan una referencia (dirección).

Variables y constantes

Variables

tipo nombre [= valor];

Si no se inicializa lo hace automáticamente: numéricas a 0, cadenas a "", boolean a false
Algunos tipos, al asignar valor hay que poner sufijos para que no den error

Sufijo	Tipo
U	unsigned
L	long
UL	unsigned long
F	Float
D	Double
M	Decimal

Constantes

const tipo nombre_constante = valor;

Conversiones

Es posible asignar el valor de un tipo más pequeño a un tipo más grande, se hace automáticamente y se llama conversión implícita. Sin embargo, la operación contraria provoca errores de compilación, se puede entonces forzar la conversión explícitamente, pero hay que asegurarse de que el valor del grande cabe en el rango del pequeño porque si no tendremos errores en ejecución

Conversiones Explícitas

- *variable* = (nuevo tipo) expresión;
- Clase Convert : implementa métodos de conversión
variable=Convert.ToDouble(variable)
- Tipo.parse(String), Tipo.TryParse

```
Int32.parse(String)
```

Comentarios

```
//, /* */
```

Operadores

- Binarios: ++,--
- Aritméticos: + , - , * , / %
- Asignación abreviada: = , += , -= , *= ,
- Concatenación : +
- Comparación == , != , < , > , <= , >=
- Lógicos: !, &&, ||

Estructuras de control

Selección: If, if – else, switch (tipos numéricos, char , string)

Bucles: while, do-while, for, foreach(variable in colección){instrucciones;}

```
foreach (Control caja in this.Controls)
    if ( caja.GetType().Name=="TextBox")
        caja.BackColor =Color.Red ;
```

Salto: break, continue

Funciones

retorno nombre (parámetros){ instrucciones;}

- Pueden pasarse parámetros por valor o por referencia. Pasar un parámetro por referencia implica no hacer copia del parámetro pasado y poder modificar su valor dentro de la función, siendo ese cambio visible fuera del ámbito de dicha función. Podemos utilizar **ref** , **out**. Si utilizo ref, el parámetro debe estar inicializado, si no lo está utilizo out.

ref	out
<pre>int n=3; Funcion_Ejemplo(ref n); //el valor de n ha cambiado</pre>	<pre>int n; Funcion_Ejemplo(out n); //el valor de n ha cambiado void Funcion_Ejemplo(out int numero){</pre>

<pre>void Funcion_Ejemplo(ref int numero){ numero++; }</pre>	<pre>numero=7; }</pre>
--	----------------------------

- Pueden pasarse un número indeterminado de parámetros, C# utiliza un array para almacenarlos. En la declaración de los parámetros de la función la sintaxis quedaría params tipo [] nombre
- Se pueden sobrecargar las funciones siempre y cuando tengan la lista de parámetros diferentes

Enumeraciones

Un enumerado es un conjunto de constantes relacionadas. Estas constantes se agrupan en un tipo con un identificador que las agrupa por la naturaleza de valores que representan: serie de valores constantes (días de la semana, colores,..)

Una enumeración deriva de System.Enum. El tipo de los valores enumerados realmente es entero, cada campo de una enumeración es estático y representa una constante a la que se le asigna un valor entero.

```
public enum Categorias  
{  
    Comercial,  
    Administrativo,  
    Secretaria,  
}
```

A cada categoría le asigna un número, empezando por 0. Se puede cambiar este valor

```
Categorias miCat= Categorias.Comercial;  
int c=(int)miCat;
```

```
int i=2;  
Categorias miCat=(Categorias)i;
```

Arrays y Matrices

```
Tipo [] nombre =new tipo[nº elementos];  
int [] lista= new int[10];
```

Para acceder a un elemento nombre[posición]
lista[0]=5;

Pueden declararse e inicializarse
int [] lista={2,3,6};

Para crear arrays bidimensionales Tipo [,] nombre=new tipo[nºfilas,nºcolumnas];

Propiedades y métodos

- Length: número total de elementos, si tiene varias dimensiones la suma de todos los elementos.
- GetLength(dimensión): número de elementos de una dimensión (empieza en 0)
- GetLowerBound(dimensión): índice inferior
- GetUpperBound(dimensión): índice superior
- Copy: para copiar elementos desde una matriz unidimensional a otra. Está sobrecargado

Array.Copy (matrizorigen, matrizdestino, nº elementos)

Array.Copy (matrizorigen, elemento inicial a copiar, matrizdestino, elemento inicial a sobrescribir, [nº de elementos])

Si no doy los índice adecuados o no tengo tamaño suficiente se produce error
Podemos usar este método para borrar elementos o insertarlos sin necesidad de recolocar todo el vector

Array.Copy (matriz, indice+1, matriz, indice, Ubound(matriz)-indice)

Luego tendría que borrar el último

Array.Copy (matriz, índice , matriz , índice +1 , matriz.length – índice – 1)

Matriz.SetValue(valor, índice)

- CopyTo: para copiar una matriz unidimensional en otra a partir de una posición en la segunda

MatrizOrigen . CopyTo (matrizdestino , índice)

- Sort : ordena los elementos de un vector

Array.Sort(matriz)

Array.Sort(matriz, índice inicial, cantidad de elementos)

- Reverse : invierte el orden de los elementos de una vector o parte de él , si se aplica después del Sort, tendremos orden descendente

Array.Reverse(matriz)

Array.Reverse(matriz, índice inicial, longitud)

Array.Reverse(matriz, índice , índice) solo intercambia esos dos valores

- Clear : borra una matriz o parte de ella (inicializa según tipo) (no elimina casillas)

Array.Clear (matriz , índice inicial, cantidad de elementos)

Array.Clear(matriz, matriz.GetUpperBound(0), 1) borramos el último elemento

- IndexOf: retorna el índice de la primera vez que aparece el valor buscado, -1 si no está. Se puede pasar la posición a partir de la que buscar.

Array.IndexOf(vector, valor)

- LastIndexOf: análogo al anterior pero retorna la última aparición

String

- Length : longitud de la cadena
- Para acceder a un caracter de una posición variable[posición]

Compare

Sobrecargado. Compara dos objetos [String](#) especificados.

CompareTo

Sobrecargado. Compara la instancia en cuestión con un objeto especificado.

Concat

Sobrecargado. Concatena una o más instancias de [String](#) o las representaciones de tipo **String** de los valores de una o más instancias de [Object](#).

Copy

Crea una nueva instancia de [String](#) con el mismo valor que una **String** especificada.

[CopyTo](#)

Copia un número especificado de caracteres situados en una posición especificada de la instancia en una posición determinada de una matriz de caracteres Unicode.

[Equals](#)

Sobrecargado. Reemplazado. Determina si dos objetos [String](#) tienen el mismo valor.

[IndexOf](#)

Sobrecargado. Devuelve el índice de la primera aparición de un objeto [String](#), o de uno o más caracteres de la instancia en cuestión.

[IndexOfAny](#)

Sobrecargado. Devuelve el índice de la primera aparición en la instancia de un carácter de una matriz de caracteres Unicode especificada.

[Insert](#)

Inserta una instancia especificada de [String](#) en una posición de índice especificada de la instancia.

[LastIndexOf](#)

Sobrecargado. Devuelve la posición de índice de la última aparición de un carácter Unicode especificado o de un objeto [String](#) en la instancia.

[LastIndexOfAny](#)

Sobrecargado. Devuelve la posición de índice de la última aparición en la instancia de uno o varios caracteres especificados de una matriz de caracteres Unicode.

[PadLeft](#)

Sobrecargado. Alinea a la derecha los caracteres de la instancia e inserta a la izquierda espacios en blanco o un carácter Unicode especificado hasta alcanzar la longitud total especificada.

[PadRight](#)

Sobrecargado. Alinea a la izquierda los caracteres de la cadena e inserta a la derecha espacios en blanco o un carácter Unicode especificado hasta alcanzar la longitud total especificada.

[Remove](#)

Elimina un número de caracteres especificado de

la instancia a partir de una posición especificada.

[Replace](#)

Sobrecargado. Reemplaza todas las apariciones de un carácter Unicode o un objeto [String](#) en la instancia por otro carácter Unicode u otro objeto **String**.

[Split](#)

Sobrecargado. Identifica las subcadenas de la instancia que están delimitadas por uno o varios caracteres especificados en una matriz, y las coloca después en una matriz de elementos [String](#).

[StartsWith](#)

Determina si el principio de la instancia coincide con el objeto [String](#) especificado.

[Substring](#)

Sobrecargado. Recupera una subcadena de la instancia.

[ToLower](#)

Sobrecargado. Devuelve una copia de [String](#) en minúsculas.

[ToString](#)

Sobrecargado. Reemplazado. Convierte el valor de la instancia en un objeto [String](#).

[ToUpper](#)

Sobrecargado. Devuelve una copia de [String](#) en mayúsculas.

[Trim](#)

Sobrecargado. Quita todas las apariciones de un conjunto de caracteres especificados desde el principio y el final de la instancia.

[TrimEnd](#)

Quita todas las apariciones de un conjunto de caracteres especificado en una matriz del final de esta instancia.

[TrimStart](#)

Quita todas las apariciones de un conjunto de caracteres especificado en una matriz del

principio de esta instancia.

DateTime

Tipo de dato para trabajar con fechas.